Totally Collectibles OS Operator Manual

Authored by David Bakanas.

Software ownership: Dag Danky Holdings LLC.

Software platform: Totally Collectibles OS (TCOS).

Main TCOS website/domain: TotallyCollectibles.com.

Flagship store / Store #1: Truely Collectables.

Last updated: 2026-06-28

This is the working manual for Totally Collectibles OS (TCOS). It must stay current as features are added.

TCOS means Totally Collectibles OS. It is the multi-store software platform, admin system, order system, inventory engine, marketplace layer, and pricing/helper system. Truely Collectables is the flagship store inside TCOS, not a separate rebuild.

Ownership And Account Separation

David Bakanas is the owner of Dag Danky Holdings LLC.

Dag Danky Holdings LLC owns and administers the Totally Collectibles OS software platform.

David Bakanas is an admin/operator of Dag Danky Holdings LLC and TCOS.

Truely Collectables LLC is the collectables storefront operating company and should be treated as Store #1 inside TCOS. It should become its own platform seller/buyer account when seller, buyer, collection, wishlist, trade, or payout accounts are built.

Dag Danky Shoes is the footwear storefront/operator for the shoe and sneaker portion of the platform. It should be treated as its own storefront seller/buyer account when footwear seller, buyer, inventory, collection, wishlist, trade, or payout accounts are built.

David Bakanas is also the admin/operator of the Truely Collectables LLC account.

Dag Danky Holdings LLC platform revenue should come from:

- platform commission/rake from third-party seller transactions
- the commission/rake is calculated from total sale amount, including item sale price plus buyer-paid shipping
- advertising revenue
- sponsorship revenue
- other platform-level service fees only after they are defined and disclosed

Truely Collectables LLC revenue and activity should stay separate from Dag Danky Holdings LLC platform revenue. Truely Collectables LLC is the storefront seller/buyer account for its own buying, selling, collecting, inventory, offers, trades, messages, payouts, and account history.

Truely Collectables LLC must have completely separate login credentials, account profile, seller settings, buyer settings, payout setup, and audit history from Dag Danky Holdings LLC platform-admin access.

Future account roles should stay separate:

- `platform_owner`: Dag Danky Holdings LLC
- `platform_admin_user`: David Bakanas and any future authorized TCOS admins
- `storefront_operator`: Truely Collectables LLC
- `storefront_seller_account`: Truely Collectables LLC selling inventory on TCOS
- `storefront_buyer_account`: Truely Collectables LLC buying/trading/collecting on TCOS when needed
- `footwear_storefront_operator`: Dag Danky Shoes
- `footwear_seller_account`: Dag Danky Shoes selling footwear inventory on TCOS
- `footwear_buyer_account`: Dag Danky Shoes buying/trading/collecting footwear on TCOS when needed
- `external_seller_account`: future third-party sellers
- `external_buyer_account`: future customers/collectors

Important rule:

Dag Danky Holdings LLC admin authority must not be mixed with Truely Collectables LLC seller/buyer activity. David may administer both, but the software should track platform-admin actions separately from seller/buyer actions so audits, payouts, disputes, chargebacks, taxes, and permissions stay clean.

Product North Star

TCOS should be built toward the ultimate collecting experience.

The collector should feel:

- confident they know exactly what the item is
- informed about rarity, demand, condition, and market value
- protected by clear policies, secure checkout, and honest evidence
- excited when the item arrives
- proud enough to share the pickup, collection, story, or mail day with the community

Every customer-facing item page should help answer:

- What exactly is this collectable?
- Why does it matter?
- How rare is it?
- What condition is it in?
- What data supports the price?
- What similar items have sold for?
- Is there population, certification, serial-number, autograph, patch, variant, or provenance information?
- What should a collector know before buying?
- What makes this item fun to own?
- What can the collector do with it after purchase, such as add it to a collection board or brag session?

Product data should be layered:

1. TCOS local inventory data
2. seller/admin-entered facts
3. AI extraction and description support
4. catalog/reference matching
5. sales comps
6. pricing guides
7. grading/certification data when available
8. rarity/population/print-run data when available
9. community context after moderation features exist

The tone should be collector-first: excited, knowledgeable, honest, and brand-safe. Public copy should invite collectors to celebrate their items without using sexual, hateful, private, or unsafe language.

Future: Collectable Megahub And Sneaker Expansion

TCOS should be designed to become a collectable megahub, not only a sports-card storefront.

Long-term ambition:

- become a premier destination for sneaker collectors
- support shoes, cards, memorabilia, comics, toys, TCG, autographs, graded items, sealed products, and other collectables
- help collectors research, find, request, buy, trade, show off, and track what they care about
- make AI search, collection tracking, and wish lists the best collector discovery experience possible
- turn Truely Collectables into a trusted place to understand collectables, not only transact on them

Sneaker marketplace goals:

- support sneaker brands such as Nike, Jordan, Adidas, New Balance, Puma, Reebok, Asics, Converse, Vans, and future brands
- support shoe model, colorway, SKU/style code, size, gender sizing, release date, condition, box status, authenticity status, defects, and provenance
- track deadstock, new with box, used, tried on, restored, custom, sample, player exclusive, collaboration, limited release, and graded/authenticated shoe states
- store multiple photos including box label, outsole, insole, size tag, heel, toe box, stitching, defects, and receipt/provenance when available
- support authentication workflow before high-value sneaker sales go public
- support size-specific pricing and comps because sneaker value changes heavily by size
- show comparable listings and sales only when the size/model/colorway match is close enough

AI collection and wish list goals:

- users can create a wish list for any collectable category
- users can describe what they want in natural language
- AI should translate wish-list language into structured search fields
- AI should match wish-list items against TCOS inventory, seller inventory, want ads, trades, auctions, and approved outside data sources

- AI should notify users only through consented notification channels
- users can mark items as grails, priority targets, set fillers, size targets, gift ideas, or research-only
- wish lists should support budget range, condition preference, grading preference, size, category, player, team, character, franchise, era, brand, colorway, and urgency

Set builder checklist goals:

- users can create checklists for complete sets, partial sets, team sets, player runs, character runs, release runs, parallel runs, graded runs, shoe colorway runs, or custom collector goals
- checklist templates should support year, brand, set, subset, card number, player, team, character, franchise, variant, parallel, serial number, autograph, relic/patch, grade target, condition target, and notes
- users can mark each checklist item as owned, needed, upgraded, incoming, watching, trade target, or not interested
- users can attach owned items from their collection to checklist slots
- users can add missing checklist items to wish lists or want ads
- AI should help build a checklist from natural language, uploaded lists, scans, set names, catalog sources, or manufacturer checklists when allowed
- TCOS should show checklist progress by percentage, count owned, count missing, estimated value when available, and highest-priority missing items
- set builders should be able to filter by missing cards, rookies, inserts, parallels, autographs, patches, graded targets, and price range

Checklist acquisition links:

- each missing checklist item should first search TCOS inventory
- if not in TCOS inventory, show approved outside lookup links
- outside links should include eBay sold/active search, CollX research, COMC search, Sportlots search, 130point sales search, Google source search, PriceCharting/SportsCardsPro, PSA when relevant, and other approved category sources
- outside links should preserve the structured checklist query as much as possible, including year, set, card number, player, team, grade, variant, size, colorway, or brand
- TCOS should clearly label outside links as external research or external marketplace links
- TCOS should not imply outside marketplaces are partners unless a partnership exists
- users should be able to save outside finds back to a checklist as `watching`, `found externally`, or `research note`

Megahub information goals:

- product pages should become research pages
- category pages should explain brands, sets, releases, pop reports, size markets, rarity, and recent demand
- collector intelligence should apply to all categories, not only cards
- buyer decisions should be supported by sources, comps, photos, condition proof, authenticity data, and clear risk notes
- AI should help summarize the collectable's story while showing source links and confidence

Competitive standard:

- TCOS should compete on trust, depth of information, buyer confidence, seller tools, AI discovery, and collector experience
- TCOS should not copy another marketplace's user experience blindly
- TCOS should avoid fake scarcity, fake hype, fake sold data, hidden fees, or misleading authenticity claims
- every expansion category should have its own data model, condition standards, authenticity rules, pricing logic, and dispute rules

Future data tables should store:

- collectable_categories
- sneaker_products
- sneaker_sizes
- sneaker_condition_reports
- sneaker_authentication_events
- sneaker_release_data
- sneaker_price_snapshots
- user_collections
- user_collection_items
- user_wish_lists
- user_wish_list_items
- wish_list_matches
- wish_list_notifications
- set_builder_checklists
- set_builder_checklist_items
- set_builder_templates
- set_builder_item_matches
- set_builder_external_links
- set_builder_progress_snapshots
- collectable_reference_sources
- collectable_category_attributes

Downloadable PDF copy:

```
docs/TCOS_OPERATOR_MANUAL.pdf
```

The PDF is regenerated with:

```
npm run manual:pdf
```

Links in this manual are clickable in both Markdown and PDF form. Use them for direct lookup, examples, API docs, and troubleshooting references.

Current V2 UI Baseline

The public storefront has been moved away from the default scaffold look.

Current UI baseline includes:

- sticky storefront navigation with TCOS/admin entry
- production metadata for Truely Collectables
- premium dark homepage hero with clear storefront positioning
- homepage feature blocks for Collector Intelligence, secure checkout, and fulfillment control
- shop page header, search/filter panel, active inventory count, and cleaner product cards
- product detail pages styled as collector research pages
- cart page with a structured order summary, shipping selector, TOS acceptance, and secure checkout action
- consistent neutral/off-white storefront background
- `/admin` now renders as a live TCOS command center with revenue metrics, fulfillment queues, offer desk, inventory watch, store settings status, evidence health, operator alerts, and fast links

1. Quick Start

Daily operator path:

1. Open `/admin/login`.
2. Log in with the admin password.
3. Open `/admin/products` to manage cards.
4. Open `/admin/orders` to ship paid orders.
5. Open `/admin/offers` to handle customer offers.
6. Use product edit pages to check comps, apply suggested prices, update descriptions, and change inventory status.

Most day-to-day work starts at:

```
/admin
```

2. Routes

Public Storefront

Route	Purpose
`/`	Home page
`/shop`	Product grid with search and sport filtering
`/product/[id]`	Product detail page
`/cart`	Customer cart
`/account`	Customer account home
`/account/login`	Customer email/password login
`/account/signup`	Customer email/password signup
`/success`	Purchase confirmation page with rotating collector sayings
`/terms`	Customer Terms of Service
`/seller-terms`	Seller Terms of Service for future auction/seller accounts

Admin

Route	Purpose
`/admin/login`	Admin login
`/admin`	Admin dashboard
`/admin/products`	Product list
`/admin/products/new`	Add product
`/admin/products/[id]`	Edit product and pricing tools
`/admin/orders`	Fulfillment center
`/admin/orders/[id]`	Order detail and tracking
`/admin/orders/[id]/packing-slip`	Printable packing slip
`/admin/files`	Transaction evidence files
`/admin/launch-readiness`	Live payment and production readiness checklist
`/admin/offers`	Offer review
`/admin/security`	Admin login audit and lockout review

API

Route	Purpose
`/api/admin/login`	Sets `admin_auth` cookie
`/api/admin/logout`	Clears admin cookie
`/api/admin/files/[id]/download`	Downloads transaction evidence PDF
`/api/account/signup`	Creates customer account through Supabase Auth
`/api/account/login`	Logs customer account in through Supabase Auth
`/api/account/orders`	Returns logged-in customer order history for the active store
`/api/checkout`	Creates Stripe checkout session
`/api/webhook`	Main Stripe webhook handler
`/api/stripe/webhook`	Alternate Stripe webhook handler
`/api/offers/create`	Customer offer submission
`/api/offers/update-status`	Accept/decline offer
`/api/offers/counter`	Send counter offer
`/api/orders/update-tracking`	Save carrier/tracking
`/api/orders/mark-shipped`	Mark shipped and send email if configured
`/api/ebay/auth`	Start eBay OAuth
`/api/ebay/callback`	Store eBay refresh token
`/api/ebay/import-listings`	Import one eBay inventory page
`/api/ebay/full-sync`	Batch import eBay inventory

3. Admin Login

Admin login uses:

```
ADMIN_PASSWORD=
```

Successful login sets a cookie:

```
admin_auth=<issued_timestamp>.<signature>
```

Cookie behavior:

- HTTP-only
- Secure

- Same-site lax
- Max age: 24 hours

Protected admin routes redirect to `/admin/login` if the cookie is missing.

Admin sessions are signed by `ADMIN_SESSION_SECRET` when configured. If `ADMIN_SESSION_SECRET` is missing, TCOS falls back to `ADMIN_PASSWORD` for signing. Production should use a separate strong `ADMIN_SESSION_SECRET`.

Admin login hardening:

- password verification uses fixed-length hashed comparison instead of plain string equality
- every login attempt is evaluated through `src/lib/admin-login-security.ts`
- failed and successful attempts are written to `admin_login_attempts` when the table is available
- login attempts store store ID, IP address, user agent, result, failure reason, identity risk, header evidence, and lockout timestamp when applicable
- five failed attempts from the same IP inside 15 minutes triggers a 15-minute lockout
- locked-out login attempts return HTTP `429`
- masked or blocked identity attempts return HTTP `403`
- if the audit table has not been migrated yet, login still works but audit/lockout storage is unavailable

Admin login policy:

Setting	Value
Failed-attempt window	15 minutes
Failed-attempt limit	5
Lockout duration	15 minutes

Admin security page:

```
/admin/security
```

This page shows recent admin login attempts, successful logins, failed logins, active lockouts, unique IP count, identity risk, failure reason, lockout expiration, and user agent. If the audit table has not been migrated yet, the page shows an unavailable warning instead of crashing.

Launch readiness also checks whether `admin_login_attempts` is available. If the table is missing or unavailable, `/admin/launch-readiness` marks Admin Login Audit as blocked.

4. Product And Inventory Basics

TCOS currently keeps two inventory layers in sync:

1. Legacy `products`
2. TCOS V2 `inventory_items`

The legacy `products` table still supports older screens and relations. The V2 `inventory_items` table is the newer inventory authority for status, price, and quantity.

The sync layer is:

```
src/modules/inventory/engine.ts
```

Do not bypass the engine for normal admin inventory changes.

Inventory V2 bridge screen:

```
/admin/inventory
```

Use this screen to verify that every legacy storefront product has a matching TCOS V2 `inventory_items` record. The page shows total legacy products, V2 bridged items, missing inventory rows, mismatch counts, active items, sold-out items, and eBay-linked items.

The `Backfill V2 Inventory` button runs an idempotent backfill. It can be run more than once. It scans Store #1 products, creates missing `inventory_items`, updates existing inventory rows by legacy product ID or SKU, mirrors quantity/price/title/description/status, and adds product images into `inventory_images` when they are not already present.

Reconciliation labels:

Label	Meaning
`OK`	Legacy product and V2 inventory row are aligned
`MISSING INVENTORY ITEM`	Product exists but no V2 inventory row exists yet
`SKU LINK ONLY`	V2 row matched by SKU but still needs the legacy product bridge filled
`QUANTITY MISMATCH`	Legacy quantity and V2 quantity differ
`PRICE MISMATCH`	Legacy price and V2 price differ
`SOLD OUT`	Product quantity is zero; not a failure by itself

eBay import safety:

The eBay import path is store-scoped. It first updates by Store #1 eBay listing ID, then by Store #1 SKU, then inserts a new product if no store-scoped match exists. It does not perform a global SKU upsert across all stores.

4A. Multi-Store Platform Foundation

TCOS is being converted from a single-store Truely Collectables app into a multi-store Totally Collectibles OS platform without breaking Store #1.

Current Store #1:

```
Truely Collectables
Truely Collectables LLC
store_id: 00000000-0000-4000-8000-000000000001
```

Foundation migration:

```
supabase/migrations/20260628110000_create_tcos_stores.sql
supabase/migrations/20260628113000_create_store_settings.sql
```

The migration:

1. Creates `stores`.
2. Inserts Store #1 = Truely Collectables.
3. Adds defaulted `store_id` columns to current core tables.
4. Backfills existing rows to Store #1.
5. Adds store indexes for future filtering.

Tables prepared with `store_id`:

- `products`
- `inventory_items`
- `orders`
- `order_items`
- `offers`
- `ebay_tokens`
- `sales_comp_snapshots`
- `tos_acceptance_events`
- `transaction_evidence_reports`

Safety rule:

Current Store #1 constants live in:

```
src/lib/stores.ts
```

Current active store context is resolved through `src/lib/stores.ts`. Store #1 is still the only active store, but app code now calls active-store helpers instead of importing the Store #1 ID across the app.

Current write paths pass the active store ID when creating products, inventory items, orders, order items, offers, eBay tokens, sales comp snapshots, TOS acceptance events, and transaction evidence reports. Current inventory, eBay token, sales comp, fulfillment, offer, evidence, admin, and success-page read/update paths are also scoped to the active store. The database default still points new rows to Store #1 as a safety net.

The inventory repository and inventory engine now carry store context internally. Future multi-store work should replace the current Store #1 resolver with request/account/domain-selected store context.

Store operational settings live in:

```
src/lib/store-settings.ts
```

The `store_settings` table stores per-store support, sales, offer, evidence, order email, Stripe mode, eBay environment, eBay account label, and seller commission settings. If the table is missing or empty, TCOS falls back to current environment variables and Store #1 defaults so production behavior does not break.

`/admin/launch-readiness` shows the active store settings and whether they came from the database or fallback defaults.

5. Inventory Status Values

Allowed status values:

Status	Meaning	Checkout allowed?
`draft`	Not ready to sell	No
`active`	Ready to sell	Yes, if quantity is enough
`reserved`	Held back	No
`sold`	Sold out	No
`archived`	Removed from active workflow	No

Checkout requires:

- status is `active`
- quantity is greater than or equal to cart quantity
- price is greater than zero

6. Add A Product

Open:

```
/admin/products/new
```

Fields:

- Title
- Player
- Sport
- Price
- Quantity
- Image URL
- Description

Description can be left blank. If blank, TCOS generates a description from product data.

When saved, TCOS:

1. Creates a row in `products`.
2. Creates a row in `inventory_items`.
3. Creates an `inventory_images` primary image row if image URL exists.
4. Redirects to `/admin/products`.

7. Edit A Product

Open:

```
/admin/products
```

Click `Edit`.

Edit page:

```
/admin/products/[id]
```

Fields editable:

- Title
- Player
- Sport
- Price
- Quantity
- Status
- Image URL
- Description

Click `Save Product`.

Save behavior:

1. Updates legacy `products`.
2. Ensures V2 `inventory_items` exists.
3. Updates V2 title/description/category/status/quantity/price.
4. If image URL changed, adds a new primary `inventory_images` row.
5. Publishes an in-memory inventory event.

8. Quick Status Buttons

On `/admin/products/[id]`:

- `Set Active`
- `Reserve`
- `Mark Sold`

- `Archive`

`Mark Sold` sets quantity to `0`.

Status changes update through `inventoryEngine.setStatus`.

9. Product Description Tools

On `/admin/products/[id]`, the Description panel has:

- `Auto-Fill Description`
- `AI Write Description`

Auto-Fill Description

Uses only local TCOS product data.

Generated from:

- title
- player
- sport
- price
- quantity
- status
- SKU
- eBay listing ID

This is deterministic and does not call outside APIs.

AI Write Description

Uses OpenAI API docs if configured:

```
OPENAI_API_KEY=
OPENAI_DESCRIPTION_MODEL=
```

Default model:

```
gpt-5.5
```

If OpenAI is not configured or the request fails, TCOS falls back to local auto-fill.

The AI prompt forbids inventing:

- year
- set
- grade
- condition

- autograph
- patch
- serial number
- rookie status
- scarcity

Product Detail Collector Intelligence

Product pages currently show a Collector Intelligence panel on:

```
/product/[id]
```

Implementation files:

```
src/app/product/[id]/page.tsx
src/lib/collector-intelligence.ts
```

Current behavior:

- lays out `/product/[id]` as a collector research page with image, status badge, price, purchase actions, description, and a collector snapshot
- the collector snapshot shows category, player/subject, availability, status, SKU, and eBay linkage when available
- shows an honest trend badge
- defaults to `Not Enough Verified Data`
- creates market research links for eBay sold search and 130point
- creates acquisition/research links for TCOS search, eBay active search, COMC, Sportlots, CollX, PriceCharting, SportsCardsPro, PSA Auction Prices, and Google marketplace search
- creates news/source links for Google News and official-source searches
- creates an X search link for the player, item, team, character, or franchise query
- shows a `Complete The Set Or Run` helper with TCOS, eBay, COMC, Sportlots, CollX, manufacturer-checklist, and 130point research links
- shows an `Exact Match Signals` panel using deterministic title extraction
- detects title-level serial numbering, card number, parallel/finish words, autograph, relic/patch, rookie, grading company, and grade signals when present
- marks title-level variant/parallel evidence as needing checklist/source confirmation before TCOS claims an exact rare variation
- detects PSA, SGC, CGC, Beckett, or BGS names in the title when present
- detects grade-like title text when present
- detects cert-number-like title text when present
- links to official grading lookup pages
- gives a watch list of things collectors should check before treating the item as hot or rare

TCOS does not currently store live trend snapshots or verified grading population counts for storefront display. Until those sources are saved and verified, the public trend state remains `Not Enough Verified Data`.

10. Sales Comps

Sales comps live on:

```
/admin/products/[id]
```

Click:

```
Check eBay Sold Comps
```

The page reloads with:

```
?comps=true
```

TCOS tries these sources:

1. eBay Marketplace Insights API
2. eBay completed-items fallback
3. Google Programmable Search JSON API, if configured
4. PriceCharting / SportsCardsPro API, if configured

TCOS also gives research links for:

- CollX
- 130point sales search
- Google sold search example
- PriceCharting example search
- SportsCardsPro
- PSA Auction Prices
- Card Ladder
- ALT
- COMC
- eBay sold search example

Lookup Examples

Use the product title, player, year, set, card number, grade, and autograph/patch terms when known.

Example searches:

- eBay sold: Michael Jordan 1991 Fleer
- Google sold-card search: Michael Jordan 1991 Fleer
- PriceCharting: Michael Jordan 1991 Fleer
- PSA Auction Prices

- 130point sales search

When checking comps, ignore listings that are not the same card or same grade. A raw card, PSA 10, BGS 9.5, autographed card, patch card, serial-numbered parallel, and base card are different markets.

11. Suggested Price

Suggested price uses a CollX-style method:

1. Average of up to 10 recent sold comps from the last six months.
2. If unavailable, use the last known sold comp.
3. If unavailable, use median of available comps.
4. If no comps exist, no suggested price is shown.

Displayed values:

- Suggested
- Count
- Median
- Average
- Range
- Recent comps used
- Source status

12. Apply Suggested Price

Click:

```
Apply Suggested Price
```

TCOS does not trust stale browser data. It recalculates comps server-side, saves a new snapshot, then updates the product price.

Flow:

1. Load current product from `inventoryEngine`.
2. Run `getSalesComps`.
3. Save a `sales_comp_snapshots` row if table exists.
4. If suggested price exists, update product through `inventoryEngine.updateProduct`.
5. Redirect back to product edit page with comps loaded.

13. Comps History

Comps history shows on product edit pages.

Table:

```
sales_comp_snapshots
```

Migration:

```
supabase/migrations/20260627160000_create_sales_comp_snapshots.sql
```

If this table is missing, the page still works but history will show a Supabase error. Apply the migration to enable saving.

14. eBay Sync

Connect eBay

Start OAuth:

```
/api/ebay/auth
```

Callback stores refresh token into:

```
ebay_tokens
```

Reference links:

- eBay Developers Program
- eBay Sell Inventory API
- eBay Marketplace Insights API

Import Listings

Single page:

```
/api/ebay/import-listings
```

Batch sync:

```
/api/ebay/full-sync
```

Import behavior:

1. Reads latest eBay refresh token.
2. Gets eBay access token.
3. Pulls eBay inventory items.
4. Fetches offer data per SKU.
5. If listing is active, updates `products` and `inventory_items`.
6. If listing is not active, marks local quantity zero.

It does not delete eBay inventory.

Current sync implementation:

```
src/lib/ebay-sync.ts
```

Both single-page import and full sync use this shared server-side importer. Full sync calls the importer directly instead of calling TCOS through its own protected HTTP route, so it does not depend on an admin browser cookie or `NEXT_PUBLIC_SITE_URL` to keep running. This keeps Truely Collectables eBay sync more reliable when the toggle is enabled.

eBay Health

Open:

```
/admin/ebay
```

This page is the local eBay reconciliation board. It does not call eBay on page load. It reads TCOS product and inventory data to show whether Store #1 inventory looks ready for eBay sync.

The page shows:

- total products
- eBay-linked products
- healthy linked products
- products needing attention
- missing SKU count
- never-synced count
- stale-sync count
- sold-out count
- latest eBay import timestamp

Health labels:

Label	Meaning
`OK`	Linked product has no local sync warning
`MISSING SKU`	Product cannot safely sync to eBay inventory without a SKU
`NOT LINKED`	Product is local-only and has no eBay listing ID
`NEVER SYNCED`	Product has an eBay listing ID but no `last_seen_at` import timestamp
`STALE SYNC`	Product has not been seen by eBay import in the configured stale window
`SOLD OUT`	Local TCOS quantity is zero

Current stale window:

```
12 hours
```

Buttons on the page:

Button	Purpose
`Test Route`	Confirms the eBay test route responds
`Import Batch`	Runs one import page from eBay
`Full Sync`	Runs the batch eBay import loop
`Reconnect`	Starts eBay OAuth again

Rule:

Use `/admin/ebay` before and after large imports. Missing SKU and stale sync warnings should be reviewed before assuming local inventory and eBay inventory agree.

eBay Sync Toggle

Open:

```
/admin/settings
```

The `Enable eBay Sync` toggle controls whether the active store can use eBay integration.

When eBay sync is enabled:

- `/api/ebay/import-listings` can import an eBay inventory page
- `/api/ebay/full-sync` can run the batch import loop
- `/api/ebay/auth` can reconnect the store's eBay OAuth token
- completed sales can push quantity changes back to eBay

When eBay sync is disabled:

- import is blocked
- full sync is blocked
- reconnect/OAuth is blocked
- post-sale eBay quantity updates are skipped
- `/admin/ebay` shows a disabled warning

Storage:

```
store_settings.metadata.ebay_sync_enabled
```

Default behavior:

```
enabled
```

This keeps Store #1 working unless an admin intentionally turns eBay sync off. For TCOS/future stores, turn this off when eBay sync should not help an outside seller or competitor.

15. Checkout

Checkout route:

```
/api/checkout
```

Customers must accept the Terms of Service before checkout can start.

Current Terms of Service:

```
/terms
```

Current TOS version:

```
2026-06-27
```

Checkout validates:

- cart is not empty
- shipping method is valid
- Terms of Service was accepted
- each product exists
- each product is `active`
- each product has enough quantity
- each product has a price greater than zero

Then it creates a Stripe checkout session.

Successful cart checkout sends the buyer to:

```
/success?type=cart&session_id={CHECKOUT_SESSION_ID}
```

Stripe metadata includes:

- cart
- shipping method
- shipping name
- shipping amount
- subtotal
- item count
- TOS accepted flag
- TOS version
- TOS accepted timestamp

Reference:

- Stripe Checkout docs

16. Stripe Webhooks

Main webhook:

```
/api/webhook
```

Alternate webhook:

```
/api/stripe/webhook
```

On `checkout.session.completed`:

1. Verify Stripe signature.
2. Create or update order.
3. Insert order items if not already inserted.
4. Decrement inventory through `inventoryEngine`.
5. Sync eBay quantity after sale.
6. If accepted offer, mark offer paid.
7. Create or update transaction evidence report.
8. Email evidence PDF if evidence email is configured.

Accepted-offer sessions use `product_id` metadata if cart metadata is missing.

Safety rule:

Inventory is decremented only after the order item row is successfully recorded. If order item insertion fails, the webhook logs the failure and does not reduce stock for that line.

Order webhooks store TOS/IP acceptance fields and create a transaction evidence report. The evidence report is intended for chargeback defense, fraud review, and legal dispute support.

Offer and counter-offer Stripe success URLs send buyers to:

```
/success?type=offer&session_id={CHECKOUT_SESSION_ID}
/success?type=counter&session_id={CHECKOUT_SESSION_ID}
```

The confirmation page uses the Stripe checkout session ID server-side to read checkout metadata, find the purchased product, and personalize the customer experience. When product data is available, it shows the purchased item image/title and themes the page with inferred team, character, franchise, or collectable colors. If product data is unavailable, it falls back to the Truely Collectables black/gold theme.

The confirmation page rotates short collector-focused sayings such as `Welcome to the family.` so the purchase feels like a meaningful addition to the customer's collection instead of a plain receipt page.

Reference:

- Stripe webhooks docs

Live Payment Readiness

Open:

```
/admin/launch-readiness
```

This page is server-rendered for admins only. It checks production configuration without exposing secret values.

It reports:

- public site URL
- Supabase configuration
- admin password/session secret configuration
- Stripe live/test/mixed key mode
- Stripe webhook secret
- IP intelligence/VPN blocking configuration
- transaction evidence email configuration
- eBay production sync configuration
- AI product helper configuration
- platform/storefront account separation status

Live buyer payments should stay closed until:

1. `NEXT_PUBLIC_SITE_URL` is the final HTTPS production domain.
2. Supabase production variables are set.
3. `ADMIN_PASSWORD` and `ADMIN_SESSION_SECRET` are set.
4. `STRIPE_SECRET_KEY` is a live key.
5. `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` is a matching live key.
6. `STRIPE_WEBHOOK_SECRET` is the live webhook signing secret for `/api/webhook`.
7. `IP_INTELLIGENCE_REQUIRED=true` and the IP intelligence provider is configured.
8. A real low-dollar checkout succeeds.
9. The order appears in admin.
10. The transaction evidence PDF is created.
11. eBay inventory quantity sync is confirmed.
12. The live test transaction is refunded in Stripe.

Platform/storefront account separation remains a readiness warning until real account roles exist. Before seller accounts, footwear operations, or third-party sellers go live, TCOS must enforce separate logins, roles, audit trails, payout profiles, and seller/buyer records for Dag Danky Holdings LLC, Truely Collectables LLC, Dag Danky Shoes, and outside sellers/customers.

17. Orders And Fulfillment

Open:

```
/admin/orders
```

Tabs:

- Ready to Ship
- Shipped
- All Orders

Order detail:

```
/admin/orders/[id]
```

Order detail shows:

- payment status
- fulfillment status
- customer name/email
- shipping address
- item list
- totals
- carrier/tracking
- packing slip link
- TOS/IP chargeback evidence
- evidence packet download if created

18. Transaction Evidence Files

Open:

```
/admin/files
```

Every completed Stripe transaction should create one evidence packet in:

```
transaction_evidence_reports
```

Evidence packets include:

- order ID
- order creation timestamp
- customer name and email
- shipping address
- carrier and tracking when saved
- shipped timestamp when marked shipped
- item list
- quantities and prices

- subtotal
- shipping paid
- total paid
- Stripe checkout session ID
- Stripe payment intent ID when available
- Stripe webhook event ID
- Stripe payment status
- accepted TOS version
- TOS accepted timestamp
- TOS acceptance audit event ID
- server-observed IP address
- user agent
- IP risk status
- IP block reason if applicable
- raw Stripe metadata saved with the transaction
- report generation timestamp

Admin actions:

- download PDF packet
- open related order
- download packet directly from the order detail page

Email behavior:

- if `RESEND_API_KEY` and `TRANSACTION_EVIDENCE_EMAIL` are set, TCOS emails the evidence PDF after the transaction report is created
- if email is not configured, the report is still saved in Admin Files
- if email fails, the error is saved on the report

Refresh behavior:

- saving tracking refreshes the saved evidence packet
- marking shipped refreshes the saved evidence packet
- the original transaction email is not resent during tracking/shipment refreshes
- the Admin Files PDF download uses the latest saved packet text

Evidence files:

```
src/lib/evidence-pdf.ts
src/lib/transaction-evidence.ts
src/app/admin/files/page.tsx
src/app/api/admin/files/[id]/download/route.ts
```

19. Tracking And Shipment

Tracking update API:

```
/api/orders/update-tracking
```

Required:

- order ID
- carrier
- tracking number

Mark shipped API:

```
/api/orders/mark-shipped
```

Requirements:

- order exists
- carrier exists
- tracking number exists

When marked shipped:

- `fulfillment_status` becomes `shipped`
- `shipped_at` is set
- the transaction evidence packet refreshes with shipment details if one exists
- email is sent if `RESEND_API_KEY` and customer email exist

Supported tracking links:

- USPS
- UPS
- FedEx

20. Packing Slips

Open:

```
/admin/orders/[id]/packing-slip
```

Use this when packing shipments.

21. Offers

Product pages include offer submission.

Customers must accept the Terms of Service before submitting an offer.

Offer creation route:

```
/api/offers/create
```

Admin page:

```
/admin/offers
```

Admin actions:

- accept
- decline
- counter

Accept/counter creates a Stripe checkout link.

Offer checkout payment decrements inventory through TCOS V2.

Accepted and counter offers must pass `inventoryEngine.requireAvailableCartItems()` before TCOS creates the Stripe checkout session. That means the product must exist, be active, have enough quantity, and have a positive checkout price.

Accepted and counter offer Stripe sessions include Store #1 metadata, cart metadata, subtotal, item count, and zero-dollar offer-checkout shipping metadata so the webhook can create a normal order record.

Accepted and counter offer Stripe sessions carry the TOS acceptance metadata from the original customer offer when available.

22. Seller Accounts And Auctions

Seller accounts and auctions are future-build features. Do not create placeholder seller tables or fake seller account workflows before the real account model is designed.

Account signup direction:

- TCOS accounts should use email and password as the baseline login method.
- Buyer/customer accounts, seller/store accounts, and platform-admin accounts must stay separate.
- Platform admin access for Dag Danky Holdings LLC must not be mixed with Truely Collectables LLC seller/buyer activity.
- Future seller accounts must have their own profile, verification status, payout-provider IDs, TOS acceptance, audit trail, and permissions.
- Future buyer accounts must have their own profile, order history, TOS acceptance, saved addresses, collection, wishlist, want ads, and communication preferences.

Current account foundation:

```
/account
/account/login
/account/signup
/api/account/login
```

```
/api/account/signup
/api/account/orders
```

Current behavior:

- customer accounts use Supabase Auth email/password
- signup requires buyer Terms of Service acceptance
- password must be at least 10 characters
- signup creates or updates `account_profiles`
- signup assigns a Store #1 buyer membership in `account_store_memberships`
- signup/login writes `account_auth_events` when the migration exists
- logged-in checkout and offer flows attach the account ID to Stripe metadata
- completed Stripe webhooks save `orders.account_id` when account metadata is present
- customer-created offers save `offers.account_id` when the customer is logged in
- `/account` shows recent linked orders for the logged-in customer
- guest checkout still works and leaves `account_id` empty
- account sessions are browser-local and separate from admin login cookies
- admin login still uses `/admin/login` and `admin_auth`

Migration:

```
supabase/migrations/20260628190000_create_tcos_accounts.sql
supabase/migrations/20260628193000_link_accounts_to_orders_offers.sql
```

Current seller legal page:

```
/seller-terms
```

Seller-account requirements:

- Truely Collectables LLC should be modeled as the storefront seller/buyer account
- David Bakanas can administer the Truely Collectables LLC account, but those seller/buyer actions must be recorded separately from Dag Danky Holdings LLC platform-admin actions
- Truely Collectables LLC must use separate login credentials and account information from Dag Danky Holdings LLC platform-admin access
- Dag Danky Shoes should be modeled as the footwear storefront seller/buyer account
- Dag Danky Shoes must use separate login credentials and account information from Dag Danky Holdings LLC platform-admin access and from Truely Collectables LLC
- sellers must accept the Seller Terms of Service before listing, auction submission, payout setup, or seller account use
- seller bank and payout information must be verified by an approved third-party payment, banking, or identity provider
- TCOS must not store raw bank credentials directly
- seller payout status must be gated by third-party verification status

- Dag Danky Holdings LLC charges a 5% seller commission/rake on third-party seller transactions
- the 5% commission is calculated from total sale amount, including item sale price plus buyer-paid shipping
- seller acceptance should be stored with seller TOS version and timestamp when seller accounts are implemented

Seller constants live in:

```
src/lib/legal.ts
```

Future: AI Collectable Scan Assist

This is a future-build feature. It must support all collectables, not only sports cards.

Goal:

- scan or upload front/back images
- use AI to identify visible details
- match the item against outside catalog/reference sources
- show confidence and possible matches
- help estimate rarity, demand, and value
- create an educated product draft only after admin approval

The system must not rely on AI guessing alone. AI detection should propose candidates, then TCOS should compare those candidates against trusted reference and marketplace sources.

Collectable categories to support over time:

- sports cards
- trading card games
- comics
- coins
- currency
- stamps
- autographs
- memorabilia
- sealed wax/product
- toys
- graded collectables
- limited edition collectables
- other cataloged collectables

Data-source research checklist:

- find official APIs first
- confirm licensing and allowed use before storing or displaying data
- prefer catalog IDs, cert numbers, set names, card numbers, issue years, print runs, population data, and verified images

- separate catalog identity data from pricing data
- record source names, source URLs, lookup timestamps, confidence scores, and raw evidence
- never auto-list a collectable from AI recognition without admin confirmation

Candidate source categories to evaluate:

- eBay sold/active marketplace data
- PriceCharting/SportsCardsPro pricing data
- PSA certification, population, CardFacts, and auction data
- TCGplayer catalog/pricing data for trading card games
- COMC card marketplace/catalog data
- Beckett/card checklist references
- manufacturer checklists when available
- grading-company certification lookups
- comic, coin, stamp, toy, and memorabilia catalog databases
- broad collectable catalog/reference sites such as Colnect-style catalogs
- Google Programmable Search as a fallback discovery layer

Scan Assist output should include:

- likely collectable title
- category
- manufacturer/brand
- year or era
- set or series
- card/comic/coin/stamp/catalog number when available
- player/character/person/franchise
- condition clues visible from scan
- grade and cert number when visible
- serial number, autograph, patch, variant, parallel, or limited mark when visible
- rarity notes
- value range with source breakdown
- confidence score
- missing information warnings
- recommended next action

Variant and parallel resolver requirements:

- TCOS should identify the exact variation or parallel whenever visual evidence and checklist data support it
- do not show a long list of near-duplicate options when the evidence resolves the match
- if the card is a refractor, TCOS should not ask whether it is pink, blue, green, gold, base, wave, mojo, cracked ice, shimmer, lava, scope, x-fractor, atomic, numbered, or unnumbered unless the scan/checklist evidence cannot prove the answer

- use front/back scan evidence, border color, foil pattern, refractor effect, serial numbering, card number, set name, year, manufacturer, player, team, logo marks, autograph/relic markers, and visible checklist identifiers
- use online checklists and manufacturer/reference sources only where access is allowed by terms or official API
- compare visual/color clues against checklist variant names and numbering ranges
- use serial numbering to narrow parallels when the checklist defines print runs
- use back-of-card codes and fine print when visible
- collapse candidates into one selected match when confidence is high and source evidence agrees
- if multiple variants remain plausible, ask one targeted question instead of showing dozens of options
- show why TCOS picked the match, including visible clues and source evidence
- show `Needs Review` instead of guessing when evidence is incomplete
- admin approval is required before auto-listing, repricing, or publicly claiming an exact rare variant

Target identification flow:

1. Extract visible facts from images.
2. Normalize title/player/team/year/set/card number/manufacturer.
3. Query approved checklist/catalog sources.
4. Compare variant names, colors, patterns, serial-number ranges, and card numbers.
5. Score candidates by evidence match.
6. Select the exact match if one candidate is clearly supported.
7. Ask one targeted follow-up if the remaining candidates differ by a detail the image cannot prove.
8. Save evidence, confidence score, source URLs, and reviewer decision.

Future data tables should store:

- scan event
- uploaded image references
- AI extracted fields
- matched catalog candidates
- selected catalog match
- source evidence
- value estimate snapshot
- admin approval decision
- variant_resolver_runs
- variant_resolver_candidates
- variant_resolver_evidence
- checklist_source_matches

Future: Expanded Product Detail Collector Intelligence

The first source-link version is implemented on product pages. The expanded version should make each collectable page feel alive by showing verified trend data, saved population-report snapshots, official social context, and current news without inventing hype or scraping restricted sources.

Target route:

```
/product/[id]
```

Goal:

- show trend signals for the player, character, franchise, team, brand, or exact collectable
- show grading population reports when a grade/cert/company is known
- link to official or approved social profiles such as X, Instagram, YouTube, team pages, manufacturer pages, or franchise pages
- show relevant news or milestones when available
- explain the item's current collector story in plain language
- help the buyer understand rarity, demand, timing, and context before purchase

Collector Intelligence panel should evaluate:

- sales velocity from TCOS/eBay/comps when available
- recent sold-price movement
- watch/search activity when available from allowed sources
- player/team news such as big games, awards, trades, records, injuries, call-ups, retirements, or playoff runs
- character/franchise news such as new movie, show, game, comic arc, anniversary, limited release, or manufacturer announcement
- grading population for the exact grade when cert/set/card details are known
- related social profiles and official links
- related collector content such as manufacturer checklist, league/team bio, player page, franchise page, or grading certification page

Trending labels must be evidence-based:

- `Trending Up` only when recent source data supports it
- `Steady Demand` when comps/activity are consistent
- `Cooling Off` when recent source data supports lower demand
- `Not Enough Data` when TCOS cannot prove a trend

TCOS must not show fake urgency, fake scarcity, fake endorsements, or unsourced claims.

Grading/population report requirements:

- support PSA, CGC, SGC, Beckett/BGS, and other grading companies only when lookup access is permitted
- prefer official cert lookup or official population data
- store grading company, grade, cert number, population count, lookup URL, lookup timestamp, and source confidence
- show `population unavailable` instead of guessing
- never imply a cert is verified unless the source confirms it

Social and news requirements:

- use official APIs, official embeds, RSS feeds, licensed news/search APIs, or manually approved links
- never bypass platform limits, scraping protections, login walls, or terms
- never imply a player, team, manufacturer, grading company, league, or franchise endorses TCOS unless there is a written sponsorship/partnership
- public social links should open to the source instead of copying restricted content into TCOS
- summarize news in TCOS language with source links and timestamps
- separate factual news from AI-generated collector commentary

AI collector commentary may help write:

- why this item is interesting
- what recently happened around the player, team, character, franchise, or brand
- what a pop report means in simple terms
- why collectors may care about the grade, parallel, serial number, autograph, patch, rookie status, set, or era

AI collector commentary must:

- cite stored sources
- say when data is missing
- avoid investment advice
- avoid guaranteed future value claims
- avoid medical, legal, or financial claims
- be reviewed or source-gated before public display

Product page display should include:

- trend badge
- short collector story
- latest relevant news link list
- official social/source links
- grading pop report card when grade/cert data exists
- last updated timestamp
- source list
- `Not Enough Data` state when sources are missing

Future data tables should store:

- collectable_intelligence_snapshots
- collectable_trend_signals
- collectable_news_links
- collectable_social_links
- collectable_grading_reports
- collectable_source_evidence
- collectable_intelligence_refresh_jobs
- collectable_intelligence_admin_reviews

Future: Brag Session And Collection Board

This is a future-build community feature. It should not copy eBay-style transactional feedback.

TCOS community ratings should use collector-style tiers:

- `Gold`
- `Silver`
- `Bronze`

These tiers should represent post-purchase collector satisfaction, community trust, and brag-session quality after moderation rules exist. They should not be implemented as a direct eBay positive/neutral/negative clone.

Goal:

- give buyers a fun way to show off what they received
- let collectors post collection photos, mail days, favorites, displays, and stories
- build community around collectables instead of only buyer/seller ratings
- keep transaction issues, chargebacks, and evidence packets separate from public community posts

Buyer post-purchase flow:

- after delivery, invite the buyer to create a `Brag Session`
- buyer can upload photos of the item or collection
- buyer can add title, story, category, tags, and optional order/item link
- buyer can choose public, private, or admin-review-only visibility
- public posts must go through moderation before appearing

Collection board:

- users can show collection shelves, slabs, binders, sealed product, memorabilia, or themed collections
- posts can be grouped by category, sport, franchise, player, set, era, or custom tags
- users can like/save/comment only after community safety controls are built
- admin can feature posts on storefront/community pages

Safety and moderation rules:

- no addresses, tracking labels, order numbers, phone numbers, payment details, or private customer information in public posts
- image uploads must be scanned/moderated before public display
- admin must be able to approve, hide, remove, or feature posts
- reporting tools should exist before public comments go live
- public community posts must not expose chargeback evidence, IP evidence, TOS audit records, or private order records
- community reputation must stay separate from seller compliance, payout, fraud, and transaction evidence workflows

Future data tables should store:

- community profile
- brag session post

- collection board post
- post images
- post moderation status
- tags/categories
- optional order item reference
- likes/saves/comments after moderation is ready
- abuse reports and admin actions

Future: Trades And Collector Swaps

This is a future-build marketplace/community feature. It should let collectors propose trades safely after buyer/seller accounts, identity controls, moderation, shipping evidence, and dispute rules exist.

Goal:

- let collectors mark items as open for trade
- let collectors build trade offers from their collection
- support one-for-one and multi-item trade proposals
- support cash difference only if payments, tax, and dispute rules are designed
- give each side a clear trade review screen before accepting
- protect both parties with identity, address, shipping, condition, and evidence controls
- keep trades separate from normal store inventory until a trade is accepted and locked

Trade section entry points to evaluate:

- `/trades` public trade browsing page
- product page `Open To Trade` signal when the owner allows it
- user collection item `Trade This` action
- want ad match to trade offer
- admin trade review queue
- post-purchase prompt to add an item to collection/trade board after delivery

Trade offer data should include:

- initiating user
- receiving user
- offered item list
- requested item list
- optional cash difference only if enabled
- item photos
- condition notes
- grade/cert details when available
- declared value from comps when available
- shipping method
- tracking numbers for both sides
- trade status

- timestamps for offer, counter, acceptance, shipment, delivery, dispute, and completion

Required safety rules:

- require user accounts before trades can exist
- require TOS acceptance for trade terms
- require verified email and recommended multi-factor authentication
- do not expose addresses until both sides accept and shipping flow requires it
- keep address and identity data private
- require item condition photos before acceptance
- record IP/user-agent evidence for trade acceptance events
- require tracking on both shipments
- allow admin freeze/review on suspicious trade activity
- prohibit off-platform payment pressure, scams, harassment, counterfeit claims, and unsafe meetups
- provide report/dispute tools before public trading goes live

Trade fulfillment models to evaluate:

- direct ship: both collectors ship to each other with tracking
- admin-mediated trade: both collectors ship to TCOS first, TCOS verifies, then forwards
- local meetup: future option only if safety, location privacy, and event rules are designed

Admin-mediated trade is safer but operationally heavier. Direct ship is easier but higher risk. TCOS should not enable public trades until the chosen model has clear rules, evidence packets, and dispute handling.

Trade evidence packets should store:

- accepted trade terms
- TOS version
- IP/user-agent evidence
- item photos at acceptance
- declared condition and value
- shipping labels/tracking
- delivery proof
- user messages related to the trade
- admin actions
- dispute notes

Future data tables should store:

- trade_profiles
- trade_items
- trade_offers
- trade_offer_items
- trade_offer_messages
- trade_acceptance_events
- trade_shipments

- trade_evidence_packets
- trade_disputes
- trade_admin_reviews
- trade_terms_acceptance

Future: Collector Map, Card Shows, And Shop Finder

This is a future-build discovery feature. It should help collectors find card shows, collectable shows, local card shops, hobby stores, trade nights, release events, grading events, and community meetups near them.

Goal:

- search by ZIP code, city, state, or current location with consent
- show nearby card shops and collectable shops on a map
- show upcoming card shows and collectable events by radius
- support geofenced regions for promoted local events
- use AI to help discover and normalize show/event information from permitted sources
- help collectors plan where to go, what to bring, and whether an event fits their collecting interests

Collector search should support:

- ZIP code
- city/state
- radius
- date range
- category, such as sports cards, TCG, comics, coins, toys, memorabilia, autographs, or mixed collectables
- event type, such as show, trade night, signing, release, grading submission event, shop event, or auction preview
- shop/event name
- free/paid admission
- family-friendly flag
- vendor/table information when available

Map and data-source candidates to evaluate:

- Google Maps Platform Places API for shop/place discovery
- Google Maps Geocoding API for ZIP/city lookup
- Mapbox or similar map provider as an alternate map/search provider
- Eventbrite or other event APIs where terms allow event discovery
- promoter-submitted event listings
- shop-owner submitted listings
- admin-entered shows and shops
- Google Programmable Search as a fallback discovery layer
- public show calendars where licensing/terms allow use
- social/event advertising sources only when access is permitted by their terms or an official API

AI event discovery should:

- parse event title, location, dates, times, promoter, admission, vendor tables, categories, and source URL
- deduplicate repeated listings for the same show
- assign confidence scores
- flag missing or conflicting details
- require admin approval before publishing uncertain events
- keep raw source evidence and lookup timestamps

Geofencing and promotion rules:

- geofenced promotions should use ZIP, city, radius, or coarse region targeting by default
- precise location should require clear user consent
- do not store exact user location unless the feature truly needs it and the user has opted in
- allow collectors to search by ZIP without enabling device location
- disclose when a result is sponsored or promoted
- keep sponsored placement separate from organic relevance
- never sell or expose private user location data

Collector-facing event pages should include:

- event/shop name
- address and map
- date/time
- distance from searched area
- event category
- admission cost
- promoter/shop contact link when available
- source link
- confidence/verification status
- last verified timestamp
- notes for collectors
- related nearby shops/events

Future data tables should store:

- shops
- shows/events
- event venues
- promoters
- event categories
- geofenced promotion campaigns
- source evidence
- verification status
- admin approvals
- user saved events
- opt-in location preferences

Future: AI Search, Want Ads, And Site Help

This is a future-build discovery and support feature.

Goal:

- help collectors find the exact item they want
- understand natural-language collector searches
- recommend matching products, categories, saved searches, shows, shops, and content
- create a Want Ad when TCOS does not have the item
- help users navigate the site and answer technical questions
- escalate support questions to a future technical support email inbox when AI cannot answer safely

AI search should support:

- player/person/character/franchise
- year, set, brand, card number, issue number, catalog number, cert number, or serial number
- sport/category
- grade, condition, raw/graded/sealed
- autograph, patch, parallel, variant, refractor, short print, rookie, insert, or limited mark
- price range
- seller-owned inventory when seller accounts exist
- active storefront products
- sold/reference/comps context when useful
- collector intent, such as buying, researching, completing a set, finding a gift, or valuing an item

Search behavior:

- return direct product matches first
- show close matches and explain why they match
- show confidence and missing details
- ask clarifying questions when the request is ambiguous
- do not invent facts or pretend an item is available
- offer to create a Want Ad if no suitable item is found

Want Ads:

- run for 30 days by default
- can be renewed by the user
- should expire automatically if not renewed
- should support status values such as active, matched, expired, renewed, canceled, and fulfilled
- should allow item details, category, desired condition/grade, budget, notes, and optional images
- should notify admin when a new Want Ad is created
- should notify the user when matching inventory appears
- should not publicly expose private email, phone, address, IP, payment, or account details
- should be moderated before public/community display if Want Ads become public

AI site help should:

- answer site navigation questions
- explain how checkout, offers, tracking, orders, TOS, evidence/security, and community features work
- guide collectors to product pages, cart, orders, Brag Sessions, Collection Board, shows/shops, and support
- avoid legal, tax, medical, financial, or authentication-sensitive claims beyond approved policy text
- avoid exposing private admin, customer, seller, payout, IP, or evidence data
- escalate unresolved technical issues to support

Technical support escalation:

- future env var should define the support inbox
- likely name: `TECHNICAL_SUPPORT_EMAIL`
- user should be able to submit name, email, issue type, order ID if relevant, and message
- AI conversation context should be summarized, not forwarded with private hidden data
- support submissions should be saved for admin review
- email forwarding should be added after the support mailbox is chosen

Future data tables should store:

- ai_search_sessions
- ai_search_messages
- want_ads
- want_ad_matches
- want_ad_renewals
- support_requests
- support_request_messages
- support_email_delivery_status

Future: Sponsorships, Advertising, And Partner Outreach

This is a future-build revenue feature. It should help TCOS sell advertising and sponsorship placements to collectable-related companies without creating spam, privacy risk, or low-quality ads.

Goal:

- create advertising inventory on TCOS
- sell logo placements, sponsored placements, newsletter spots, show/shop placements, and category sponsorships
- help collectable companies reach collectors through relevant pages
- create a compliant sponsor outreach workflow
- track leads, conversations, packages, contracts, payment status, creative assets, start/end dates, and performance

Potential sponsor categories:

- card shops
- card show promoters
- grading companies

- supplies companies
- storage/display companies
- auction houses
- breakers and stream sellers
- collectable insurance companies
- memorabilia companies
- comic, coin, toy, stamp, and TCG businesses
- local shops promoted by ZIP, city, state, radius, or event category

Ad inventory to evaluate:

- homepage sponsor block
- shop/category sponsor block
- product-page related sponsor block
- Collector Map sponsor pins
- card show/event sponsor placement
- Brag Session/Collection Board sponsor placements after moderation tools exist
- newsletter/email sponsor slot after subscriber consent tools exist
- admin-approved banner/logo placements

Sponsor packages should define:

- placement
- audience/category
- region or geofence
- price
- duration
- impressions/clicks when tracking is built
- creative/logo requirements
- prohibited content
- approval workflow
- renewal rules

Outreach rules:

- do not scrape or harvest email addresses in ways that violate laws, terms, or platform rules
- do not send deceptive mass email
- use truthful sender identity, subject lines, and offer language
- include Dag Danky Holdings LLC / Truely Collectables contact information
- include a valid physical mailing address when sending commercial outreach
- include a clear opt-out/unsubscribe method
- honor opt-out requests within 10 business days
- keep a suppression list and never email suppressed contacts again except as required for compliance
- separate one-to-one relationship outreach from marketing blasts

- prefer warm leads, opt-in lists, business cards, show contacts, inbound sponsor forms, and manually verified public business contacts

TCOS can help with:

- sponsor media kit copy
- rate card
- sponsorship package definitions
- compliant outreach email templates
- lead tracker
- outreach status pipeline
- follow-up reminders
- opt-out/suppression tracking
- sponsor asset uploads
- ad placement scheduling
- invoice/payment tracking
- performance reporting after analytics is built

TCOS should not send sponsor outreach until:

- `SPONSOR_OUTREACH_FROM_EMAIL` is configured
- `SPONSOR_OUTREACH_REPLY_TO_EMAIL` is configured
- `SPONSOR_OUTREACH_PHYSICAL_ADDRESS` is configured
- opt-out handling is built
- suppression list handling is built
- a reviewed outreach template is approved
- contacts are sourced and verified through allowed methods

Reference:

- FTC CAN-SPAM Act Compliance Guide

Future data tables should store:

- sponsors
- sponsor_contacts
- sponsor_leads
- sponsor_outreach_messages
- sponsor_outreach_suppression_list
- sponsor_packages
- ad_placements
- ad_creatives
- ad_campaigns
- ad_impressions
- ad_clicks
- sponsor_invoices
- sponsor_payments

Future: Marketing Campaigns, Social Posting, And Collectable Of The Day

This is a future-build promotion feature. It should help TCOS advertise the website, promote listings, and keep social channels active without spam, fake engagement, deceptive posting, or platform policy problems.

Goal:

- create organized advertising campaigns for TCOS
- schedule approved promotional posts across connected social channels
- automatically feature a Collectable of the Day
- generate platform-specific captions, hashtags, and safe tags
- promote store listings, card shows, sponsor campaigns, collection posts, want ads, and major inventory drops
- track campaign status, approvals, channels, post history, clicks, conversions, and engagement
- avoid repetitive posting, unwanted tagging, or random website posting that looks like spam

Supported campaign types to evaluate:

- Collectable of the Day
- new inventory drop
- featured category
- local card show or shop spotlight
- sponsor spotlight
- holiday/event promotion
- price drop
- buyer brag/collection highlight after customer permission
- want ad spotlight
- newsletter promotion after subscriber consent tools exist

Collectable of the Day automation should:

- select one eligible product per day from active inventory
- avoid sold, hidden, draft, or blocked products
- prefer items with strong photos, clean title, price, category, and description
- create a short caption for each connected platform
- generate hashtags from product title, category, brand, manufacturer, player, team, set, year, grade, sport, franchise, rarity, and condition
- tag manufacturers, teams, players, leagues, or brands only when the tag is accurate, relevant, and not excessive
- avoid tagging companies or public figures repeatedly just to force attention
- include the product link and clear call to action
- support admin preview and approval before publishing
- record exactly what was posted, where it was posted, and when it was posted

Posting controls should include:

- enabled channels
- posting calendar

- daily/weekly posting limits
- quiet hours
- duplicate detection
- campaign approval status
- link tracking
- image selection
- generated caption review
- generated hashtag review
- generated tag review
- failure/retry handling
- admin override

Web and community promotion rules:

- do not randomly post to unrelated websites
- do not bypass moderation, captchas, posting limits, or site rules
- do not use bots to create fake engagement, fake comments, or fake accounts
- post only where TCOS has permission, an account, an official integration, or clear allowed community participation
- respect each site's terms, community rules, rate limits, and promotional posting policies
- keep a record of where TCOS is allowed to post and what kind of promotion is allowed there
- prefer official APIs, approved social integrations, owned channels, paid advertising accounts, newsletters, and permission-based communities

Social channels to evaluate:

- Facebook page and groups where promotion is allowed
- Instagram business account
- TikTok business account
- X account
- YouTube Shorts
- Pinterest
- LinkedIn company page for sponsor/business updates
- Reddit only in communities where promotional posting is allowed by the subreddit rules
- Discord servers only where the server owner permits promotions
- email newsletter after subscriber consent tools exist

Campaign copy should be generated in the TCOS voice:

- collector-first
- accurate
- excited but not misleading
- no fake scarcity
- no fake endorsements
- no unsupported price claims

- no unauthorized claim that a player, manufacturer, team, league, or brand sponsors TCOS

TCOS should not publish automated posts until:

- at least one official social account is connected
- platform API access or posting method is approved
- admin approval workflow is built
- duplicate/spam controls are built
- channel-specific rules are saved
- link tracking is configured
- image/caption/tag review is enabled
- opt-in newsletter consent exists for email campaigns

Future data tables should store:

- marketing_campaigns
- marketing_campaign_channels
- marketing_campaign_posts
- marketing_campaign_assets
- marketing_campaign_post_approvals
- marketing_campaign_post_metrics
- collectable_of_the_day_candidates
- collectable_of_the_day_posts
- social_accounts
- social_channel_rules
- social_posting_limits
- social_post_failures
- campaign_link_clicks

23. Security And Data Protection

Security is required for owner, buyer, and seller data.

Current implemented protections:

- admin sessions use a signed HTTP-only cookie instead of a plain `true` cookie
- admin session cookies are secure, same-site lax, and expire after 24 hours
- site usage can be blocked when VPN, proxy, Tor, relay, hosting, or anonymous IP risk is detected
- blocked site users see: `Sorry, you must turn off your proxy or VPN to use this website.`
- admin pages are protected by `src/proxy.ts`
- sensitive admin-style API routes are protected by the same admin session
- security headers are applied globally from `src/proxy.ts`
- admin/API responses are marked `no-store`
- Stripe webhooks require Stripe signature verification
- customer TOS acceptance is required before checkout and offer submission

- customer TOS acceptance records the server-observed public IP, user agent, IP risk, and request header evidence
- checkout creates a `tos_acceptance_events` audit row before Stripe checkout is created
- completed transactions create `transaction_evidence_reports` for chargeback/legal packets
- offer submission stores TOS/IP evidence before the offer is accepted
- bank credentials are not stored in TCOS

Important IP rule:

TCOS can log and enforce the server-observed public client IP. If a customer hides behind a VPN, proxy, Tor, relay, or hosting network, TCOS cannot discover the hidden residential IP by itself. Production masking detection requires a third-party IP intelligence provider configured through environment variables.

Routes currently protected by admin session:

- `/admin/*`
- `/ebay/*`
- `/api/admin/*` except `/api/admin/login`
- `/api/ebay/*`
- `/api/orders/*`
- `/api/offers/update-status`
- `/api/offers/counter`

Public routes that must remain public:

- `/shop`
- `/product/[id]`
- `/cart`
- `/terms`
- `/seller-terms`
- `/api/checkout`
- `/api/offers/create`
- `/api/webhook`
- `/api/stripe/webhook`

Webhook routes are exempt from the site-wide identity gate so Stripe can deliver signed webhook events.

Required future seller-account protections:

- use a real identity/auth provider for buyer and seller accounts
- require email verification before buying or selling account use
- require multi-factor authentication for sellers and admins
- use a third-party provider for bank verification and payouts
- store only provider IDs, verification status, timestamps, and non-sensitive payout metadata
- never store raw bank account numbers, routing numbers, SSNs, tax IDs, passwords, or payment card data in TCOS
- encrypt sensitive internal notes or documents if they are ever added
- restrict seller data so one seller cannot read another seller's account, payout, auction, or customer data

- audit seller-account changes, payout changes, bank verification changes, and admin overrides
- rate-limit login, checkout, offer, and seller onboarding endpoints before public launch
- add fraud review and payout hold controls before seller payouts go live

Security files:

```
src/lib/admin-session.ts
src/lib/client-identity.ts
src/lib/tos-acceptance.ts
src/proxy.ts
```

24. Clean Fake Orders

To clear fake local orders, run this in Supabase SQL Editor:

```
truncate table order_items, orders restart identity cascade;
```

This clears only local orders.

It does not delete:

- products
- inventory
- eBay listings
- eBay inventory
- offers
- eBay tokens

If fake orders reduced quantity, run eBay sync afterward.

25. Environment Variables

Core:

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
NEXT_PUBLIC_SITE_URL=
```

Admin:

```
ADMIN_PASSWORD=
ADMIN_SESSION_SECRET=
```

Stripe:

```
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
```

eBay:

```
EBAY_CLIENT_ID=
EBAY_CLIENT_SECRET=
EBAY_ENVIRONMENT=production
```

Email:

```
RESEND_API_KEY=
TRANSACTION_EVIDENCE_EMAIL=
TRANSACTION_EVIDENCE_FROM=
TECHNICAL_SUPPORT_EMAIL=
```

`TRANSACTION_EVIDENCE_EMAIL` is the fallback destination address for transaction evidence PDFs when `store_settings.evidence_email` is not set. `TRANSACTION_EVIDENCE_FROM` is optional and defaults to the store evidence sender. `TECHNICAL_SUPPORT_EMAIL` is the fallback support inbox when `store_settings.support_email` is not set.

AI descriptions:

```
OPENAI_API_KEY=
OPENAI_DESCRIPTION_MODEL=
```

Optional comps:

```
GOOGLE_SEARCH_API_KEY=
GOOGLE_SEARCH_ENGINE_ID=
PRICECHARTING_API_TOKEN=
```

IP intelligence and masked-identity blocking:

```
IP_INTELLIGENCE_API_URL=
IP_INTELLIGENCE_API_KEY=
IP_INTELLIGENCE_REQUIRED=true
```

`IP_INTELLIGENCE_API_URL` may contain `{ip}` as a placeholder. If `IP_INTELLIGENCE_REQUIRED=true`, TCOS blocks site/API usage when the provider is missing, unavailable, or reports VPN/proxy/Tor/hosting/anonymous risk.

Reference links:

- Vercel environment variables
- Supabase project settings

- Stripe docs
- eBay developer keys
- Google Programmable Search JSON API
- OpenAI API docs

26. Database Tables

Legacy compatibility:

- `products`
- `orders`
- `order_items`
- `offers`
- `ebay_tokens`

TCOS V2:

- `inventory_items`
- `inventory_images`
- `inventory_attributes`
- `sales_comp_snapshots`
- `tos_acceptance_events`
- `transaction_evidence_reports`

See:

```
docs/DATABASE_DOCUMENTATION.md
```

27. Migrations

Migration directory:

```
supabase/migrations
```

Current migration:

```
20260628193000_link_accounts_to_orders_offers.sql
20260628190000_create_tcos_accounts.sql
20260628180000_create_admin_login_attempts.sql
20260628114000_create_inventory_tables.sql
20260628113000_create_store_settings.sql
20260628110000_create_tcos_stores.sql
20260627160000_create_sales_comp_snapshots.sql
20260627170000_add_tos_acceptance_to_orders_offers.sql
```

```
20260627173000_add_tos_identity_evidence.sql
20260627180000_create_transaction_evidence_reports.sql
```

Apply migrations before using features that depend on new tables.

Reference:

- Supabase database migrations

28. Build And Verification

Run:

```
npm run build
```

Expected:

- compile succeeds
- TypeScript succeeds
- route generation succeeds

Use this before deploy or after feature changes.

Reference:

- Next.js docs

29. Safe Operating Rules

Do:

- use admin status buttons instead of deleting products
- use eBay sync to restore stock from eBay
- check comps before repricing important cards
- apply suggested price only after reviewing comps
- update tracking before marking shipped

Do not:

- delete eBay tokens casually
- delete inventory rows manually
- assume clearing orders restores quantity
- assume AI descriptions know card facts not entered in TCOS
- scrape pricing sites without permission/API

30. Troubleshooting

Admin redirects to login

Cookie is missing or expired. Log in again.

Checkout says not enough inventory

Check product status and quantity on `/admin/products/[id]`.

Product does not show in shop

Check:

- status is `active`
- quantity is above zero
- price is above zero

eBay sync fails

Check:

- `EBAY_CLIENT_ID`
- `EBAY_CLIENT_SECRET`
- `ebay_tokens` has a refresh token
- eBay API scopes are valid

Sales comps do not save

Apply:

```
supabase/migrations/20260627160000_create_sales_comp_snapshots.sql
```

Google comps show not configured

Set:

```
GOOGLE_SEARCH_API_KEY=
GOOGLE_SEARCH_ENGINE_ID=
```

PriceCharting shows not configured

Set:

```
PRICECHARTING_API_TOKEN=
```

AI description falls back to local

Check:

```
OPENAI_API_KEY=
OPENAI_DESCRIPTION_MODEL=
```

31. Developer Map

Inventory:

```
src/modules/inventory
```

eBay, comps, pricing:

```
src/lib/ebay.ts
```

Checkout:

```
src/app/api/checkout/route.ts
```

Terms of Service:

```
src/lib/legal.ts
src/app/terms/page.tsx
supabase/migrations/20260627170000_add_tos_acceptance_to_orders_offers.sql
```

Stripe webhooks:

```
src/app/api/webhook/route.ts
src/app/api/stripe/webhook/route.ts
```

Transaction evidence:

```
src/lib/evidence-pdf.ts
src/lib/transaction-evidence.ts
src/app/admin/files/page.tsx
src/app/api/admin/files/[id]/download/route.ts
supabase/migrations/20260627180000_create_transaction_evidence_reports.sql
```

Admin product screens:

```
src/app/admin/products/page.tsx
src/app/admin/products/new/page.tsx
src/app/admin/products/[id]/page.tsx
```

Orders:

```
src/app/admin/orders
src/app/api/orders
```

Offers:

```
src/app/admin/offers
src/app/api/offers
```

32. Maintenance Rule

When a feature changes, update this manual in the same work session.

PDF generation can wait until the end of a completed module or lane so development can move faster. Keep the Markdown manual current during implementation, then run `npm run manual:pdf` at the module checkpoint.

Checklist for future changes:

1. Update feature behavior section.
2. Update route list if routes changed.
3. Update environment variables if new keys are added.
4. Update database docs if tables/fields change.
5. Run `npm run build`.

The app should not get ahead of the documentation.